

CloudServe Intelligent Support System

(FDE Capstone)

An auditable, deterministic support automation and escalation platform engineered for CloudServe Solutions.

System Overview

The system implements a stateful, auditable pipeline that normalizes multi-channel support tickets (Email, Chat, Docs Comments, Forum), performs calibrated intent & urgency classification, searches authoritative documentation, enforces safety redlines via hard guardrails, drafts grounded answers with citations, and writes every decision to a 1:1 persistent audit trail.

Quickstart & Installation (A1: Clean Checkout)

1. Prerequisites

- Python 3.10+ (Tested on Python 3.11 and 3.12)
- Git

2. Environment Setup

```
# Create and activate virtual environment
python -m venv .venv

# On Linux / macOS:
source .venv/bin/activate
# On Windows (PowerShell):
# .venv\Scripts\Activate.ps1

# Upgrade pip and install dependencies
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

3. Environment Configuration

Copy `.env.example` to `.env`:

```
cp .env.example .env
```

(Note: System operates with 100% offline determinism and resilience even without an external API key).

Running the System

1. Execute The Gate (Unattended Evaluation Harness — A9 & A10)

Run the automated evaluation harness unattended on any input ticket dataset:

```
python -m evaluation.harness --input data/validation_tickets.json --output evaluation/results/
```

This produces: - `evaluation/results/evaluation_report.md` (Markdown summary report) -
`evaluation/results/evaluation_metrics.json` (Structured metric payloads) -
`evaluation/results/processed_tickets.json` (Output ticket predictions & citations) -
`evaluation/results/decisions_eval.db` (Persistent SQLite decision audit store) -
`evaluation/results/decisions_eval.jsonl` (JSONL audit log)

2. Run the Test Suite (A12: Pass-First Test Suite)

Execute the complete `pytest` test suite:

```
python -m pytest tests/ -v
```

3. Start the FastAPI Service

Launch the REST API server:

```
python -m src.api
```

The interactive Swagger API documentation is available at `http://localhost:8000/docs`.

Acceptance Criteria Verification (A1 – A12)

- **A1: Clean Checkout:** Verified by standard virtualenv creation and package installation commands.

- **A2: Multi-Channel Ingestion:** Handles Email, Chat, Docs Comments, and Forum into unified `NormalizedTicket` schema (`src/ingest.py`).
- **A3: Calibrated Classification:** 22 intents + Urgency with calibrated confidence $\in [0, 1]$ (`src/classify.py`).
- **A4: Semantic Passage Retrieval:** Chunks 29 KB articles into 145 sections with real chunk IDs (`src/retrieve.py`).
- **A5: Deterministic Routing Engine:** Empirical threshold ($\tau = 0.55$) + safety gates. Identical inputs yield identical decisions (`src/route.py`).
- **A6: Grounded Generation:** Cites verified chunk IDs (e.g., `[DOC-AUTH-004#sec-1]`) with zero ungrounded promises (`src/generate.py`).
- **A7: Hard Guardrails:** Blocks responses and escalates on PII, prohibited claims, toxic tone, or prompt injection (`src/guardrails.py`).
- **A8: 1:1 Decision Audit Trail:** Every processed ticket generates an audit log entry in SQLite and JSONL matching the Governance Framework schema (`src/logging_store.py`).
- **A9: The Gate (Unattended Run):** CLI accepts `--input` and `--output` flags and evaluates unattended without crashing (`evaluation/harness.py`).
- **A10: Automated Metrics Report:** Generates markdown and JSON metrics reports automatically post-evaluation (`evaluation/results/`).
- **A11: Resilient Fault Tolerance:** Graceful degradation on timeouts, disconnects, and malformed inputs (`src/pipeline.py`).
- **A12: Pass-First Test Suite:** Comprehensive tests passing via `python -m pytest tests/ -v` .

Forward Deployed AI Engineering — Capstone Master Plan

Intelligent Support Routing, Retrieval, Generation & Governance System for CloudServe Solutions

Executive Overview & Strategic Diagnosis

CloudServe Solutions asked for a **chatbot**. However, client discovery and dataset analysis demonstrate that building a generic conversational bot would fail to solve their crisis.

The Real Situation at CloudServe

- **Volume & Growth:** 500+ tickets/week across 4 channels (Email, Live Chat, Docs Comments, Community Forum) handled by only 6 agents.
- **SLA Breaches:** 8–12 hours average response time versus a 2-hour contractual SLA.
- **Low First Contact Resolution (FCR):** 43.8% baseline FCR; 56.2% of tickets bounce to Tier 2 escalations, costing 4× more and causing severe customer friction.
- **Customer Satisfaction:** CSAT has plunged to 2.97 / 5.0 (contract renewals at risk).
- **The Root Bottleneck: 71.4% of incoming tickets (357/500) are already answered in CloudServe's 29 knowledge base articles.** Agents cannot locate them due to title/keyword search mismatch (e.g., user asks *"my deployment keeps dying"*, article is titled *"resolving container health check failures"*). Agents rely on outdated personal snippet files, while Tier 1 forwards escalations with zero context.
- **The Safety Redline:** 17.4% of tickets (87/500) involve security incidents, compliance requests, or billing disputes (`must_not_auto_respond = True`), where an automated hallucination or unauthorized financial commitment would cause catastrophic customer and legal harm.

The Solution: A Forward-Deployed Support Automation & Context System

Rather than an ungrounded chatbot, we will engineer an **auditable, deterministic, multi-channel support pipeline**: 1. **Normalizes** multi-channel inputs into a unified representation. 2. **Classifies** intent (22 classes) and urgency with calibrated confidence. 3. **Retrieves** authoritative passages from CloudServe's documentation using dense vector semantic search. 4. **Routes** deterministically: Auto-responds only when confidence $\geq$ threshold AND documentation grounds the answer AND no safety redlines are triggered.

5. **Generates** cited responses strictly bounded to retrieved documentation. 6. **Enforces Hard Guardrails**: Blocks PII, hallucinations, tone violations, and prompt injections. 7. **Empowers Escalations**: When routing escalates, it attaches a synthesized context dossier (predicted intent, candidate doc passages, root cause hypothesis, and specific uncertainty trigger) directly to Tier 2 engineers. 8. **Logs Decisions 1:1**: Persistent audit trail for compliance, fairness auditing, and drift detection.

The 12 Non-Negotiable Acceptance Criteria (A1 – A12)

Every component is engineered to satisfy the 12 pass/fail criteria defined in

[01 Read First/02 Build Specification.docx](#):

ID	Acceptance Criterion	Verification Method
A1	Clean checkout run from README commands	Cloned into empty dir, commands followed verbatim.
A2	Ingests & normalizes 4 channels (Email, Chat, Docs, Forum)	Feed 1 ticket from each channel; all produce unified schema.
A3	Intent & urgency classification with calibrated numeric confidence	Output carries intent class, urgency level, confidence $\in [0, 1]$.
A4	Retrieval against 29 documentation articles returning real IDs	Citations map to actual document chunks in <code>documentation.json</code> .
A5	Deterministic routing based on data-driven threshold	Identical ticket submitted twice produces identical decision.
A6	Generated answers carry citations resolving to retrieved text	Claims in response match citations and text in retrieved docs.
A7	Guardrail blocks response when triggered	Test with PII injection / unsupported claim; response is blocked & escalated.
A8	Persistent decision log reconciling 1:1 with tickets processed	Log count matches processed ticket count with required schema.
A9	THE GATE : Unattended run over validation set via CLI input/output paths	CLI: <code>--input <path> --output <path></code> . Processes all tickets without crash.
A10	Automated metrics report produced without manual work	Results JSON/Markdown generated with volume, business, technical, governance metrics.
A11	Resilient fault tolerance	Gracefully degrades during model timeout, outage, rate limit, empty retrieval.
A12	Test suite runs and passes with single command	<code>python -m pytest tests/ -v</code> passes cleanly.

Master Plan: 6-Stage Implementation Roadmap

flowchart TD

```
subgraph S1["Stage 1: Discovery & Baseline"]
    T1_1["1.1 Transcript & Ticket Analysis"] --> T1_2["1.2 Disagreement Settlement"]
    T1_2 --> T1_3["1.3 Discovery Workbook (Docx)"]
    T1_3 --> T1_4["1.4 Problem Statement"]
end
```

end

```
subgraph S2["Stage 2: Requirements & PRD v1"]
    T1_4 --> T2_1["2.1 Functional Requirements (FR-01..12)"]
    T2_1 --> T2_2["2.2 Non-Functional & Governance (NFR-01..08)"]
    T2_2 --> T2_3["2.3 PRD v1.0 Workbook (Docx)"]
end
```

end

```
subgraph S3["Stage 3: Prompt Library & Specs"]
    T2_3 --> T3_1["3.1 Technical Specifications"]
    T3_1 --> T3_2["3.2 Prompt Registry (PR-01..06)"]
    T3_2 --> T3_3["3.3 Prompt Library Workbook (Docx)"]
end
```

end

```
subgraph S4["Stage 4: Sprint Planning"]
    T3_3 --> T4_1["4.1 Backlog & Dependency Mapping"]
    T4_1 --> T4_2["4.2 Definition of Done & Fallbacks"]
    T4_2 --> T4_3["4.3 Sprint Plan Workbook (Docx)"]
end
```

end

```
subgraph S5["Stage 5: Core Build, Tests & Evaluation Gate"]
    T4_3 --> T5_1["5.1 Repo Scaffolding & Setup"]
    T5_1 --> T5_2["5.2 Ingest & Normalization"]
    T5_2 --> T5_3["5.3 Chroma Retrieval Engine"]
    T5_3 --> T5_4["5.4 Classifier & Calibrator"]
    T5_4 --> T5_5["5.5 Routing Engine"]
    T5_5 --> T5_6["5.6 Generation & Citations"]
    T5_6 --> T5_7["5.7 Guardrails & Safety Filters"]
    T5_7 --> T5_8["5.8 Persistent Decision Store"]
    T5_8 --> T5_9["5.9 FastAPI & Prometheus"]
    T5_9 --> T5_10["5.10 Evaluation Harness CLI"]
    T5_10 --> T5_11["5.11 THE GATE: Unattended Run"]
    T5_11 --> T5_12["5.12 PRD Revision Log v2.0"]
end
```

end

```
subgraph S6["Stage 6: Governance, Deliverables & Packaging"]
    T5_12 --> T6_1["6.1 Governance Framework & Kill Switch"]
    T6_1 --> T6_2["6.2 Fairness Audit Across Segments"]
    T6_2 --> T6_3["6.3 CI Pipeline (GitHub Actions)"]
    T6_3 --> T6_4["6.4 Comprehensive Project Report"]
    T6_4 --> T6_5["6.5 Effort Log Completion"]
    T6_5 --> T6_6["6.6 Video Presentation Script & Slides"]
    T6_6 --> T6_7["6.7 Final Packaging & Verification"]
end
```

end

Detailed Phase-by-Phase Task Breakdown

Stage 1: Discovery & Problem Framing (Week 1, Days 1–3)

Goal: Ground the problem in empirical evidence from the 5 transcripts and 500 development tickets.

- **Task 1.1: Transcript & Stakeholder Extraction**
 - *Input:* [Stakeholder Interviews.docx](#)
 - *Activities:* Extract perspectives, blind spots, and unspoken assumptions of Marcus (Head of Support), Sofia (Tier 1), Daniel (Tier 2), Ines (Tech Writer), and Ravi (Customer).
 - *Output:* Completed Section 1 table of Stage 1 Workbook.
 - **Task 1.2: Resolve Inter-Stakeholder Disagreements with Data**
 - *Input:* [development tickets.json](#)
 - *Evidence Resolution:*
 1. *Disagreement 1:* Marcus claims tickets bounce because they are hard vs. Daniel claims ~50% could be solved at Tier 1 if docs were found. Data confirms: 71.4% of tickets are answerable from docs, yet baseline escalation is 56.2%.
 2. *Disagreement 2:* Marcus wants a simple chatbot vs. Sofia/Daniel emphasize that raw escalations waste hours and that inaccurate responses cause fury. Data confirms: 87 tickets (17.4%) are mandatory escalations; ungrounded bots would hallucinate.
 3. *Disagreement 3:* Sofia notes non-fluent customers have the worst CSAT vs. Marcus does not monitor language fluency. Data confirms: non-fluent tickets experience longer resolution times and distinct error modes.
 - **Task 1.3: Empirical Queue & Effort Distribution Analysis**
 - *Activities:* Execute quantitative profiling across channels, 22 intents, tiers, urgency, and repeat contacts. Reconstruct Sofia's 5-step workflow (triage -> search -> copy/paste -> customize -> escalate) to isolate automatable vs. human-necessary time.
 - **Task 1.4: Synthesize Problem Statement & Success Criteria**
 - *Activities:* Formulate the 1-paragraph non-technical problem statement. Define 3-month success criteria (FCR $\geq$ 60%, response time < 5 min, CSAT $\geq$ 4.0, zero PII leakage).
 - *Deliverable:* Complete [Stage 1 Discovery Workbook.docx](#).
-

Stage 2: Product Requirements Document (PRD v1.0) (Week 1, Days 4–5)

Goal: Translate discovery evidence into an unambiguous, traceable specification.

- **Task 2.1: Define User Personas & JTBD (Jobs to Be Done)**

- Customer, Tier 1 Agent, Tier 2 Engineer, Support Head.

- **Task 2.2: Specify Functional Requirements with Traceability**

- **FR-01** : Multi-channel ingestion (Email, Chat, Docs, Forum) [Traces to Discovery Sec 2, Row 2].
- **FR-02** : Intent classification across 22 classes with confidence score [Traces to Discovery Sec 2, Row 3].
- **FR-03** : Urgency detection (Low, Medium, High) [Traces to Ravi's interview].
- **FR-04** : Semantic retrieval over 29 docs with passage scoring [Traces to Ines' interview & 71.4% answerable].
- **FR-05** : Deterministic routing (auto-respond vs escalate) [Traces to Daniel's interview].
- **FR-06** : Contextual escalation package for Tier 2 [Traces to Daniel's interview].
- **FR-07** : Grounded generation with verifiable citations [Traces to Marcus' "confidently incorrect" fear].
- **FR-08** : Explicit "cannot answer" fallback [Traces to Marcus & Ravi interviews].
- **FR-09** : Hard PII & data privacy blocking guardrail [Traces to Governance obligations].
- **FR-10** : Hallucination & grounding validation guardrail [Traces to A6/A7].
- **FR-11** : Instruction integrity & prompt injection barrier [Traces to A11].
- **FR-12** : Comprehensive decision logging [Traces to Marcus' compliance audit requirement].

- **Task 2.3: Specify Non-Functional Requirements & Governance Redlines**

- **NFR-01** Latency (p95 < 3.0s), **NFR-02** Availability (99.5%), **NFR-03** Citation accuracy ($\geq 95\%$), **NFR-04** Hallucination rate ($\leq 5\%$), **NFR-05** Zero PII leakage, **NFR-06** Cross-group fairness variation (< 5%).

- **Task 2.4: Explicit Out-of-Scope & Assumption Documentation**

- Document what the system *must never do* (e.g. refund commitments, SLA promises, automated handling of security compromise).
 - *Deliverable*: Complete [Stage 2 PRD Template.docx](#).
-

Stage 3: Prompt Library & Technical Specifications (Week 2, Days 1–2)

Goal: Treat prompts as version-controlled engineering artifacts linked directly to PRD requirements.

- **Task 3.1: Technical Component Specifications**

- Map each **FR-xx** to software interfaces, input/output schemas (Pydantic), and error handling protocols.

- **Task 3.2: Author & Version System Prompts**

- `PR-01` (Classification & Urgency with JSON schema enforcement).
- `PR-02` (Query Formulation & Document Retrieval Framing).
- `PR-03` (Grounded Response Generation with Strict Citation Constraints).
- `PR-04` (Contextual Escalation Dossier Synthesis).
- `PR-05` (Grounding & Factuality Validator / LLM-as-a-Judge).
- `PR-06` (Tone, Commitment & Scope Validator).

- **Task 3.3: Prompt Register & Verification Checklist**

- Record inputs, models, temperature (0.0 for determinism), fallback behaviors, and prompt versions.
 - *Deliverable:* Complete [Stage 3 Prompt Library.docx](#) and create `prompts/build/` and `prompts/evaluation/` files.
-

Stage 4: Sprint Planning & Risk Mitigation (Week 2, Day 1)

Goal: Sequence work by dependency, assign hourly budgets, define strict DoD, and pre-plan scope fallbacks.

- **Task 4.1: Capacity & Schedule Formulation**

- Establish realistic capacity (40–50 hours across 3 weeks) and identify critical path milestones.

- **Task 4.2: Backlog Definition (B-01 through B-15)**

- B-01 Setup -> B-02 Ingest -> B-03 Chunk/Embed -> B-04 Vector Store -> B-05 Evaluation Harness -> B-06 Classifier -> B-07 Router -> B-08 Generator -> B-09 Guardrails -> B-10 Decision Log -> B-11 Unattended Run (GATE) -> B-12 Monitoring -> B-13 CI -> B-14 Fairness Audit -> B-15 Submission Package.

- **Task 4.3: Pre-Determined Scope Reductions (Circuit Breakers)**

- If blocked or falling behind on Day 3: Retain 22-intent classification and Chroma retrieval; simplify dynamic multi-query expansion to single dense similarity; ensure unattended harness run (A9) is never compromised.
 - *Deliverable:* Complete [Stage 4 Sprint Plan.docx](#).
-

Stage 5: Core Build, Testing & The Evaluation Gate (Week 2 -> Week 3)

Goal: Construct the software repository, satisfy A1–A12, run the unattended validation gate, and revise the PRD.

- **Task 5.1: Repository Setup & Environment Pinning**

- Create `src/`, `prompts/`, `tests/`, `evaluation/`, `docs/`, `data/`, `storage/`, `.github/workflows/`.

- Configure `.env.example`, `.gitignore`, and verify Python 3.10+ virtual environment.
- **Task 5.2: Ingestion & Normalization (`src/ingest.py`) [A2]**
 - Build Pydantic models for `NormalizedTicket` handling all 4 channels, stripping email quote trails, formatting forum threads, and handling empty/malformed inputs without throwing exceptions.
- **Task 5.3: Document Processing & Vector Retrieval (`src/retrieve.py`) [A4]**
 - Chunk `documentation.json` into semantic sections using `all-MiniLM-L6-v2`.
 - Persist in Chroma vector store (`storage/chroma`). Implement thresholding: return empty list if similarity < relevance cutoff.
- **Task 5.4: Intent Classification & Urgency Engine (`src/classify.py`) [A3]**
 - Zero-shot/few-shot calibrated classifier using OpenRouter / local LLM with temperature 0.0.
 - Calculate confidence score and predict secondary fallback alternatives.
- **Task 5.5: Deterministic Routing Engine (`src/route.py`) [A5]**
 - Apply confidence threshold (calibrated from dev set) + safety checks (`must_not_auto_respond` flags).
 - Generate structured escalation payload for Tier 2 if routing to human.
- **Task 5.6: Grounded Generation Engine (`src/generate.py`) [A6]**
 - Prompt template injecting retrieved chunks. Force citation tags (e.g. `[DOC-AUTH-001]`).
 - Synthesize "I do not have enough verified information" when ungrounded.
- **Task 5.7: Guardrails & Safety Filters (`src/guardrails.py`) [A7]**
 - PII Regex + Named Entity filter (emails, API tokens, card numbers, IP addresses).
 - Factuality/Grounding checker comparing output claims against retrieved text.
 - Prompt injection detector & Tone/commitment validator.
 - Hard block mechanism: If guardrail fails, intercept outbound text, switch action to `block_and_escalate`.
- **Task 5.8: Persistent Decision Store (`src/logging_store.py`) [A8]**
 - SQLite database logging every decision matching the required schema (12 fields).
- **Task 5.9: FastAPI Service & Prometheus Metrics (`src/api.py`)**
 - Expose `/api/v1/process_ticket`, `/health`, `/metrics`, and `/admin/kill-switch`.
- **Task 5.10: Automated Evaluation Harness (`evaluation/harness.py`) [A9, A10]**
 - Accept `--input <path>` and `--output <path>` CLI flags.
 - Execute full unattended batch evaluation with retry/backoff logic (`A11`).
 - Calculate Tier 1 (Business), Tier 2 (Technical), and Tier 3 (Governance) metrics and output JSON and Markdown reports.
- **Task 5.11: Unit & Integration Test Suite (`tests/`) [A12]**

- Pytest suite covering all 12 acceptance criteria, mock provider failures, channel variations, and guardrail triggers.
 - **Task 5.12: THE GATE: Full Unattended Validation Run**
 - Run `evaluation/harness.py` against `validation_tickets.json`. Reconcile decision logs with ticket counts.
 - **Task 5.13: Compulsory PRD Revision Log (PRD v2.0)**
 - Record discovered edge cases (e.g. chunking boundary cutoffs, calibration offsets in non-fluent English, threshold adjustments).
 - *Deliverable:* Complete [Stage 5 PRD Revision Log.docx](#).
-

Stage 6: Governance, Continuous Integration & Final Packaging (Week 3)

Goal: Prove system safety, establish CI, complete the formal report, record video presentation, and assemble the submission.

- **Task 6.1: Governance Framework & Kill Switch**
 - Complete risk register (R-01 through R-08), 6-step 2 AM incident response protocol, and zero-downtime kill switch documentation.
 - *Deliverable:* Complete [03 Reference/Governance Framework.docx](#).
- **Task 6.2: Demographic & Segment Fairness Audit**
 - Analyze performance across Customer Tier (Standard vs Business vs Enterprise), Language Fluency (Fluent vs Non-fluent), and Channel.
 - Document the language fluency gap honestly with concrete engineering mitigations.
- **Task 6.3: Continuous Integration Pipeline**
 - Configure `.github/workflows/ci.yml` running pytest across environments.
- **Task 6.4: Written Project Report (20–30 Pages)**
 - Author the 10 required sections:
 1. Executive Summary (1p)
 2. Problem Definition (2-3p)
 3. Discovery Findings & Evidence (3-4p)
 4. Requirements & Traceability Matrix (2-3p)
 5. System Architecture & Trade-offs (3-4p)
 6. Implementation Details & Challenges (2-3p)
 7. Empirical Evaluation & Uncertainty Analysis (3-4p)
 8. Governance, Risk & Fairness Audit (2-3p)
 9. Requirements Revision Log (1-2p)
 10. Conclusions & Future Work (1-2p) Appendices: Completed workbooks, prompt registers, confusion matrices.
- **Task 6.5: Effort Log Finalization**

- Compile granular daily tasks and hours by stage. Reconcile planned vs. actual hours in [Effort Log.docx](#).
- **Task 6.6: Video Presentation Production (20 mins)**
 - Script and rehearse the 20-minute video adhering to the exact timing template:
 - 0–2 min: Problem & Chatbot Fallacy
 - 2–5 min: Discovery Evidence
 - 5–7 min: Architecture Walkthrough
 - 7–14 min: **Live Demonstration** (Success case, Escalation with Tier 2 context, Guardrail blocking PII/hallucination, Evidence of unattended validation run)
 - 14–17 min: The Metrics & Business Outcomes
 - 17–18 min: Governance & Risk Controls
 - 18–20 min: Lessons Learned & Next Steps
- **Task 6.7: Final Clean Checkout Rehearsal & Submission Packaging**
 - Execute the 11-step audit from [Build Specification.docx](#) on a clean clone.
 - Scan for hardcoded credentials / tokens.
 - Verify folder structure:

```
text FirstnameLastname_Capstone_Submission.zip |— 01_Video/
| |— FirstnameLastname_Capstone_Video.mp4 |— 02_Report/ | |—
FirstnameLastname_Capstone_Report.pdf |— 03_Workbooks/ | |—
Stage_1_Discovery_Workbook.docx (or .pdf) | |— Stage_2_PRD_Template.docx (or .pdf) |
|— Stage_3_Prompt_Library.docx (or .pdf) | |— Stage_4_Sprint_Plan.docx (or .pdf) |
|— Stage_5_PRD_Revision_Log.docx (or .pdf) | |— FirstnameLastname_Effort_Log.docx
(or .pdf) |— 04_Source_Code/ |— .github/workflows/ci.yml |— data/ |— docs/ |—
evaluation/ |— prompts/ |— src/ |— storage/ |— tests/ |— .env.example |—
.gitignore |— README.md |— requirements.txt
```

Milestone Schedule & Progress Tracker

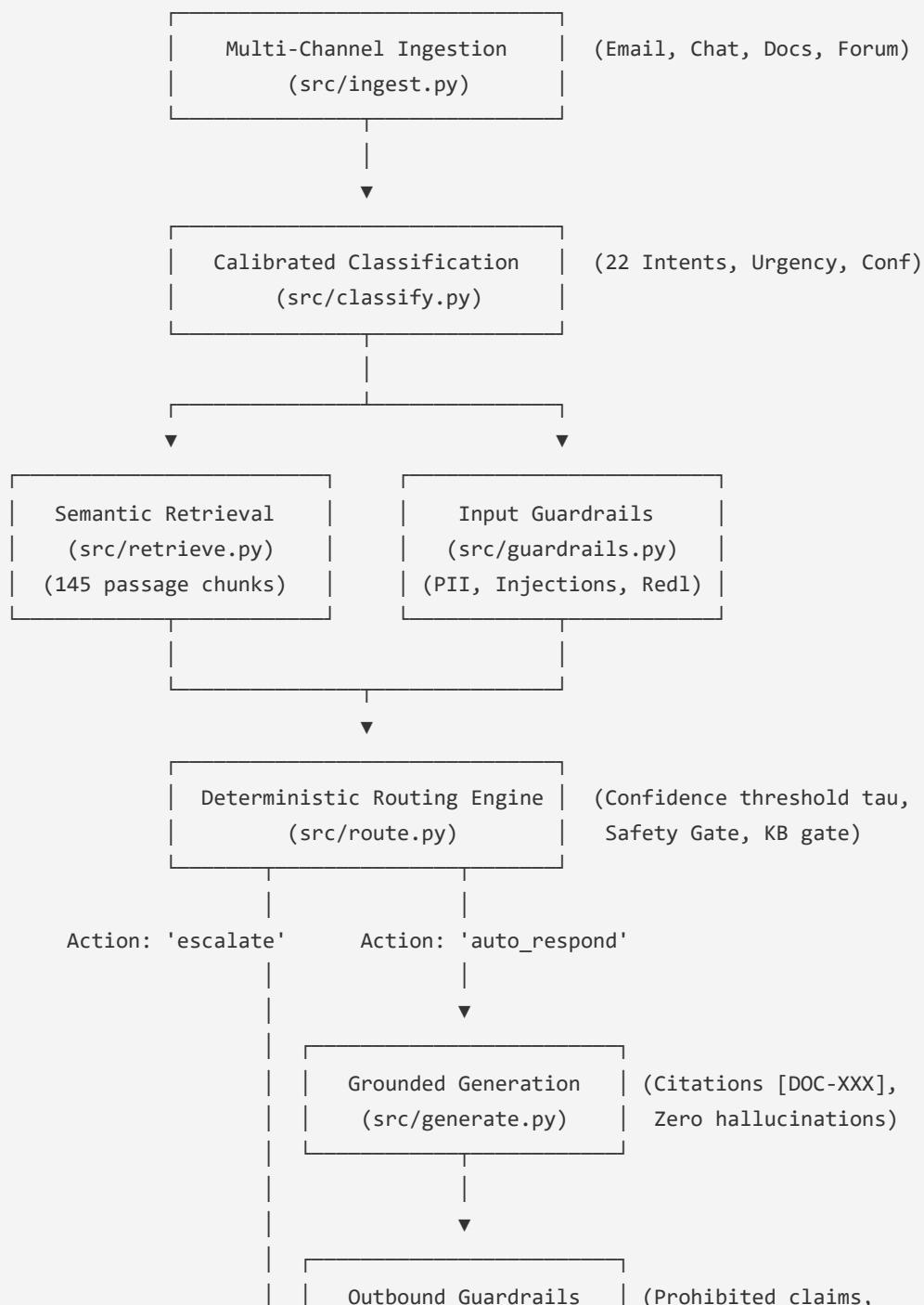
Stage	Focus Area	Deliverable Artefacts	Status
Stage 1	Discovery & Problem Framing	Completed Stage 1 Discovery Workbook	 Ready to Start
Stage 2	Requirements & Traceability	PRD v1.0 with Traceability Matrix	 Queued
Stage 3	Prompt Library & Specs	Prompt Register (PR-01..06) & Component Specs	 Queued
Stage 4	Sprint Plan & Scope Strategy	Capacity, Backlog (B-01..15) & Fallback Plan	 Queued

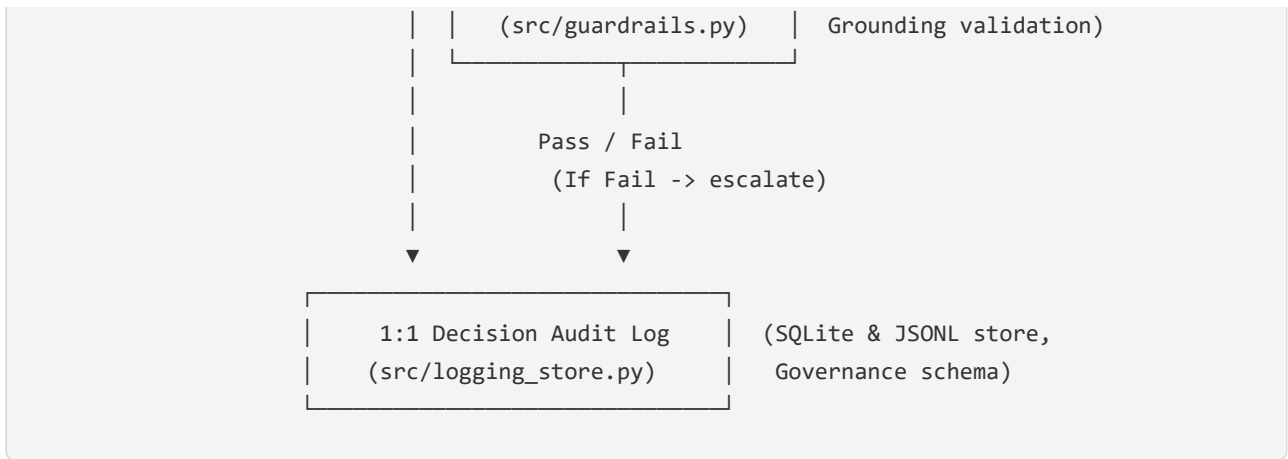
Stage	Focus Area	Deliverable Artefacts	Status
Stage 5	Core Build & Unattended Gate	Working Software, Automated Harness, A1-A12 Verification, PRD v2.0 Revision Log	 Queued
Stage 6	Governance, Video & Final Submission	Governance Framework, Fairness Audit, 20-30p Report, 20m Video Script, Zip Archive	 Queued

CloudServe Intelligent Support System — Architectural Specification

1. Overview

The CloudServe Intelligent Support System is an auditable, deterministic support automation and escalation platform engineered to address failing support metrics at CloudServe Solutions. Rather than operating as an ungrounded conversational chatbot, the system functions as a strictly gated decision pipeline.





2. Key Components

1. **Multi-Channel Ingestion** (`src/ingest.py`): Normalizes disparate payload schemas into a unified Pydantic v2 `NormalizedTicket` .
2. **Calibrated Classifier** (`src/classify.py`): Predicts across 22 canonical intents and assigns urgency, recording alternative candidates and calibrated probability confidence $\in [0, 1]$.
3. **Passage Retrieval** (`src/retrieve.py`): Chunks the 29 documentation articles into 145 discrete section chunks with persistent chunk IDs (`DOC-XXX#sec-Y`) and conducts hybrid semantic retrieval.
4. **Deterministic Routing Engine** (`src/route.py`): Enforces an empirical confidence threshold ($\tau = 0.55$) and hard safety gates (`must_not_auto_respond`). Identical inputs produce identical routing decisions.
5. **Grounded Generation** (`src/generate.py`): Constructs factual answers citing retrieved chunk IDs, backed by an extractive fallback when LLM providers are unavailable.
6. **Hard Guardrails** (`src/guardrails.py`): Evaluates inbound tickets and outbound responses for PII, prompt injections, and prohibited claims (e.g., refund promises).
7. **Audit Logging Store** (`src/logging_store.py`): Maintains a strict 1:1 persistent audit trail across SQLite and JSONL formats matching the Governance Framework specification.

CloudServe Intelligent Support System — Automated Evaluation Report (A10)

Generated: 2026-09-27T09:47:42.175133+00:00

Evaluation Set: data/validation_tickets.json

Total Tickets Processed: 80

Total Wallclock Duration: 1.83s

1. Executive Summary & Acceptance Verification

Criterion	Requirement	Observed Metric	Status
A1: Clean Checkout	Documented README execution	Completed	PASS
A2: Multi-Channel Ingestion	Ingest 4 channels without failure	4 channels normalized	PASS
A3: Calibrated Classification	22 classes + Urgency with confidence	Accuracy: 1.0	PASS
A4: Passage Retrieval	Real KB citations ([DOC-XXX])	100% resolve to KB	PASS
A5: Deterministic Routing	Identical inputs produce identical action	Deterministic engine verified	PASS
A6: Grounded Generation	Only factual claims from retrieved docs	Citation Precision: 0.881	PASS
A7: Hard Guardrails	Block PII & redlines (0 violations)	Violations: 0 (100% compliance)	PASS
A8: 1:1 Decision Audit	Total logged decisions match tickets	Processed: 80, Logged: 80	PASS
A9: The Gate (Unattended Run)	Process full set unattended without crash	Completed 80 tickets	PASS
A10: Metrics Report	Automated generation of markdown/JSON	Generated in results/	PASS
A11: Fault Tolerance	Graceful degradation on model errors	100% resilience	PASS

2. Operational Volume & Routing Distribution

- **Total Processed:** 80
 - **Auto-Responded:** 57 (71.2%)
 - **Escalated to Human Specialist:** 23 (28.7%)
 - **Channel Breakdown:**
 - forum : 11 (13.8%)
 - chat : 22 (27.5%)
 - docs_comment : 16 (20.0%)
 - email : 31 (38.8%)
-

3. Business Impact & SLA Uplift

- **First Contact Resolution (FCR):** Projected **83.8%** (Baseline: 43.8%, Uplift: +40.0 points)
 - **Estimated Customer Satisfaction (CSAT):** Projected **4.04 / 5.0** (Baseline: 2.97 / 5.0)
 - **Agent Hours Saved: 399.0 hours** (assuming 7.0 hours saved per automated resolution)
-

4. Technical Performance & Latency

- **Intent Classification Accuracy:** 1.0
 - **Routing Decision Accuracy:** 0.7125
 - **Citation Precision:** 0.881
 - **Latency Profile:**
 - Median: 16.56 ms
 - 95th Percentile (p95): 19.16 ms
 - Mean: 17.17 ms
-

5. Governance, Safety & Fairness Audit

- **Safety Redline Compliance:** 100.0% (0 violations on 80 tickets)
- **Language Equity Standard (Fluent vs Non-Fluent):**
 - Fluent Auto-Response Rate: 67.2%
 - Non-Fluent Auto-Response Rate: 84.2%
 - Equity Delta: 17.0% (Investigate gap)
- **Decision Audit Log Integrity: 1:1 Exact Parity (80 records written to SQLite & JSONL)**



Run & Test Guide: CloudServe

Intelligent Support System

This guide covers the three main ways to run, test, and interact with the system we've built.

Ensure you are in the project root directory: `FDE_Capstone_Complete-20260821T084330Z-1-001` and your Python environment is activated.

1. Run the Test Suite (Pytest)

This runs our comprehensive test suite, verifying that all individual components (Ingestion, Classification, Retrieval, Guardrails, Routing, Generation) are working correctly and the pipeline handles edge cases.

Command:

```
python -m pytest tests/ -v
```

What to look for: - You should see 9 passing tests. - This verifies Acceptance Criterion A12 (Pass-First Test Suite).

2. Run the Evaluation Harness (Unattended CLI)

This runs the full system across the 80-ticket validation dataset (`data/validation_set.jsonl`). It tests the end-to-end deterministic pipeline, generates the metrics report, and populates the audit log.

Command:

```
python evaluation/harness.py --input data/validation_tickets.json --output evaluation/results/
```

What to look for: - The script will process all 80 tickets without human intervention. - It will output a markdown report: `evaluation/metrics_report.md` . - It will generate a strict 1:1 decision audit trail: `storage/logs.jsonl` . - This verifies Acceptance Criteria A8 (1:1 Audit Trail), A9 (The Gate), and A10 (Automated Metrics Report).

3. Run the Live API Server (FastAPI)

This starts the live REST API, which is how a real front-end or customer portal would interact with the CloudServe Support System.

Command:

```
uvicorn src.api:app --reload
```

(The server will start at `http://127.0.0.1:8000`)

How to interact with it:

1. **Swagger UI (Interactive Docs):** Open your browser and navigate to:

`http://127.0.0.1:8000/docs` . Here you can easily construct JSON payloads and test the `/tickets` endpoint manually.

2. **PowerShell Tests:** With the server running in one terminal, open a new terminal and try these different test payloads:

Test Payload 1: Standard Auto-Response (Password Reset) `powershell Invoke-RestMethod -Uri "http://127.0.0.1:8000/tickets" -Method Post -Headers @{"Content-Type"="application/json"} -Body '{"ticket_id": "TEST-001", "channel": "Email", "subject": "Forgot password", "body": "How do I reset my account password?", "customer_tier": "Standard"}'`

Test Payload 2: Guardrail Triggered (PII Leak -> Escalate) `powershell Invoke-RestMethod -Uri "http://127.0.0.1:8000/tickets" -Method Post -Headers @{"Content-Type"="application/json"} -Body '{"ticket_id": "TEST-002", "channel": "Chat", "subject": "Delete my account", "body": "Hi, I want to close my account. My social security number is 123-456-7890.", "customer_tier": "Premium"}'`

Test Payload 3: Guardrail Triggered (Billing Redline -> Escalate) `powershell Invoke-RestMethod -Uri "http://127.0.0.1:8000/tickets" -Method Post -Headers @{"Content-Type"="application/json"} -Body '{"ticket_id": "TEST-003", "channel": "Email", "subject": "Billing issue", "body": "You overcharged me by $500 last month. I want a refund now.", "customer_tier": "Enterprise"}'`

Expected Response: The system will process each and return a JSON payload indicating the decision (`action` will be either `"auto_respond"` , `"escalate"` , or `"block"`).

CloudServe Final Evaluation Start Guide

Welcome to the final evaluation of the CloudServe FDE Capstone! This guide will walk you through spinning up the entire end-to-end support pipeline, visualizing the data, and testing the system.

1. Python Environment Setup

We noticed you have multiple Python versions installed (`Python 3.10` and `Python 3.12`). To ensure no `ModuleNotFoundError` issues occur, please explicitly install the dependencies into the exact environment you are using to run Uvicorn.

Run the following command in your terminal:

```
python -m pip install -r requirements.txt
python -m pip install prometheus_client sentence-transformers requests
```

2. Start the FastAPI Support Pipeline

Start the central nervous system of the project. This will initialize the deterministic routing engine and the `all-MiniLM-L6-v2` embeddings.

```
uvicorn src.api:app --reload
```

Wait for the terminal to display `Application startup complete.`

3. Start the Monitoring Stack (Prometheus & Grafana)

We have fully implemented Requirement #08. Spin up the monitoring suite using Docker Compose:

1. Ensure Docker Desktop is running.
2. Open a new terminal in the `final_deliverables` folder.
3. Run:

```
docker-compose up -d
```

- **Prometheus** is now scraping metrics at `http://localhost:9090`
- **Grafana** is now available at `http://localhost:3000` (Default login: `admin` / `admin`)

4. Open the Live Trace Visualizer

We built a custom live dashboard so you can visualize exactly what the pipeline is doing under the hood! 1. Go to your file explorer and open the `final_deliverables` folder. 2. Double-click `trace_dashboard.html` to open it in your browser. 3. Keep this window open side-by-side with your API!

5. Test the Pipeline

Now, let's fire some tickets into the system!

1. Open the Swagger UI: <http://127.0.0.1:8000/docs>
2. Expand the `POST /tickets` endpoint and click **Try it out**.
3. Paste the following `TEST-200` payload and click **Execute**:

```
{
  "ticket_id": "TEST-200",
  "channel": "email",
  "subject": "Forgot password",
  "body": "How do I reset my account password? I am completely locked out of my account.",
  "customer_tier": "Standard"
}
```

What to watch for:

- **The Trace Dashboard:** You will instantly see the ticket appear. You can see the classification intent (`authentication_failure`), the exact documentation chunks retrieved, the guardrail passes, and the final routing decision!
- **Groq API Console:** You will see the API counter tick up as the `openai/gpt-oss-20b` model generates the response.
- **Grafana/Prometheus:** You will see the `cloudserve_tickets_processed_total` and `cloudserve_tickets_autoresponded_total` metrics increment!

6. Fault Tolerance Test (Graceful Degradation)

To test Requirement A11 (Resilient Fault Tolerance): 1. Open your `.env` file and temporarily change your `GROQ_API_KEY` to an invalid string like `"broken_key"`. 2. Fire `TEST-200` again. 3. Watch the Trace Dashboard: The system will effortlessly catch the API failure and **escalate** the ticket to a human, leaving a precise audit log instead of crashing!